

Overview

Track 16.3 –  LangChain · 16-ai-tech-stack/tracks/langchain/README.md

LangChain — Module 16.3

Where you are: Track 3 of the AI tech-stack module, between `../python/` and `../langgraph/`. **Time:** ~8–10 hours · **Prereq:** Python (type hints, async basics), one LLM API call under your belt, `../.../04-llm/`.

LangChain is the framework everyone has an opinion about and comparatively few people can describe accurately — partly because it rewrote itself twice while they were forming the opinion. This track teaches the 1.x framework: what `langchain-core` actually gives you, why the packages split, where LCEL still belongs, and — in the same breath — when the right answer is to delete the dependency and call the provider SDK directly.

Everything here runs **offline with no API key**. The chat models are hand-rolled fakes, so you can read the output and know it came from the code in front of you rather than from a model's mood.

Learning objectives

By the end of this track you can:

1. State, in two sentences, what LangChain is for and give three concrete situations where using it is the wrong call — including the abstraction and debugging costs you are accepting.
2. Draw the package graph (`langchain-core` → `langchain` → `langgraph`, plus provider / community / classic packages) and say which packages you would pin, and to what.
3. Implement a `BaseChatModel` subclass from scratch and explain why `.batch()`, `.astream()`, `.with_retry()` and `.with_structured_output()` then work on it without further code.
4. Compose an LCEL chain using `RunnableParallel`, `RunnablePassthrough.assign`, `configurable_fields`, `with_retry` and `with_fallbacks` — and say what each one costs at runtime.
5. Write the tool-calling loop by hand, and handle a hallucinated tool name, a `ToolException`, and schema-invalid arguments without crashing the request.
6. Build a RAG chain from loader → splitter → embeddings → vector store → retriever → prompt → model, and demonstrate the grounded-looking hallucination it produces on an out-of-corpus question.
7. Diagnose a broken chain from a trace: name the six failure modes in §5 of the craft note and their fixes.
8. Decide, with stated criteria, whether a given feature belongs in LCEL, in `create_agent`, or in hand-written `LangGraph`.

Suggested path

#	Do this	File	Time
1	Read the core concepts	notes/01-core-concepts.md	1.5h
2	Read the fake-model source before the examples	code/fakes.py	20m
3	Run and read the LCEL tour	code/lcel_basics.py	1h
4	Run the tool-calling + structured-output lab	code/tools_and_structured_output.py	1h
5	Run the offline RAG chain	code/mini_rag_chain.py	1h
6	Read the production half	notes/02-practitioner-craft.md	1.5h
7	Do the exercises	lab/EXERCISES.md	3h
8	Skim the primary sources	papers/PAPERS.md	1h
9	Present it back	slides/ (langchain.pptx)	30m

Run everything with the shared venv:

```
/Users/srinip/ai-all/.venv/bin/pip install -r code/requirements.txt
cd code && /Users/srinip/ai-all/.venv/bin/python lcel_basics.py
```

The 16 terms you must own

Runnable · LCEL · RunnableSequence · RunnableParallel · RunnablePassthrough · RunnableBinding ·
invoke/batch/stream/astream · astream_events · with_retry · with_fallbacks · configurable_fields ·
bind_tools · ToolMessage / tool_call_id · with_structured_output · checkpointer / thread_id ·
langchain-classic

Exit check

Take `code/mini_rag_chain.py` and extend it so that:

1. the retriever drops any chunk below a similarity threshold you chose *and justified in a comment*;
2. when nothing survives the threshold, the chain abstains without calling the model at all;
3. the answer comes back as a pydantic model with `answer: str | None` and `sources: list[str]`, and a `ValidationError` degrades gracefully instead of raising;
4. a `pytest` test proves all of the above against the fake model, with **no network access**, in under a second.

Then write one paragraph arguing either for or against keeping LangChain in that pipeline at all. If the paragraph is a fair fight, you have understood the track.